Practical No. 09

AIM: Load Testing Using Apache JMeter
Design and execute load testing scenarios for a web application. Configure thread groups, ramp-up time, and loop count. Analyze response time, throughput, and error percentage using graphical reports.

Tool: Apache JMeter

Theory -
Load Testing Using JMeter
Apache JMeter is an open-source tool used for performance and load testing of web applications and APIs. It simulates multiple users accessing an application at the same time.

JMeter helps measure response time, throughput, and error rate under different loads. It is commonly used to identify performance problems and determine how an application behaves when many users access it simultaneously.

Steps for Load Testing:
Step 1 - Open Apache JMeter on Windows
Go to the folder where Apache JMeter was extracted.
For example: D:\STQA\apache-jmeter-5.6.3\bin\ ApacheJMeter.jar

Step 2 - Create a New Test Plan
When JMeter opens, you will see:
Test Plan
Step 1
Right-click **Test Plan**.
Select:Add
→ Threads (Users)
→ Thread Group
You will now have:
Test Plan
└── Thread Group

Step 3 - Configure the Thread Group
Click **Thread Group**.
You will see:

- Number of Threads (users)
- Ramp-up period (seconds)

- Loop Count

For **Test 1**, enter:

Parameter	Value
Number of Threads	10
Ramp-up Period	10
Loop Count	5

Therefore: 10 Users × 5 Loops = 50 Requests
assuming there is one HTTP request in the loop.

Step 4 - Add HTTP Request
Now we need to specify which web application should be tested.
Step 1
Right-click **Thread Group**.
Select:Add
→ Sampler
→ HTTP Request

The structure becomes:
Test Plan
└── Thread Group
└── HTTP Request

Step 5 - Configure the HTTP Request
Click **HTTP Request**.
Suppose your authorized test application is:
https://example.com
Enter the following:

Field	Value
Protocol	https
Server Name or IP	example.com
Port Number	Leave blank
Method	GET
Path	/

Step 6 - Add Summary Report
We need a report to analyze the test results.
Step 1
Right-click **Thread Group**.

Select:
Add
→ Listener
→ Summary Report

Your structure becomes:

```
Test Plan
└── Thread Group
    ├── HTTP Request
    └── Summary Report
```

Step 7 - Add Aggregate Report
Right-click **Thread Group**.
Select:
Add
→ Listener
→ Aggregate Report
Now:

```
Test Plan
└── Thread Group
    ├── HTTP Request
    ├── Summary Report
    └── Aggregate Report
```

The Aggregate Report is useful for:

- Average response time
- Median response time
- Minimum response time
- Maximum response time
- 90th percentile
- 95th percentile
- 99th percentile
- Error percentage
- Throughput

Step 8 - Add Response Time Graph
Right-click **Thread Group**.
Select:
Add
→ Listener
→ Response Time Graph
This will allow you to visualize response time.

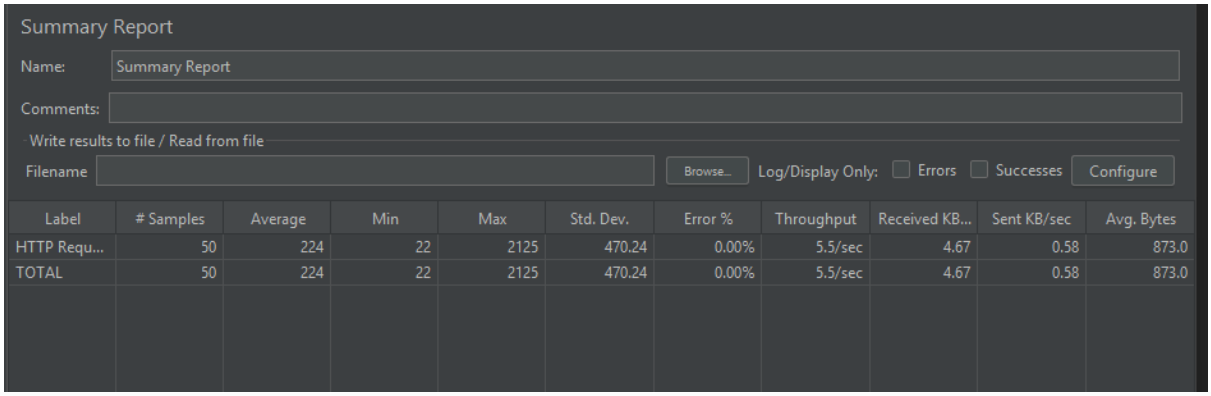
Step 9 - Add Graph Results
Right-click **Thread Group**.
Select:
Add
→ Listener
→ Graph Results
Your final basic structure should look like:
Test Plan

```
└── Thread Group
    ├── HTTP Request
    ├── Summary Report
    ├── Aggregate Report
    ├── Response Time Graph
    └── Graph Results
```

Step 10 - Save the Test Plan
Click:
File → Save Test Plan As
Save the file as: LoadTesting21.jmx
For example: D:\STQA\apache-jmeter-5.6.3\bin\LoadTesting21.jmx

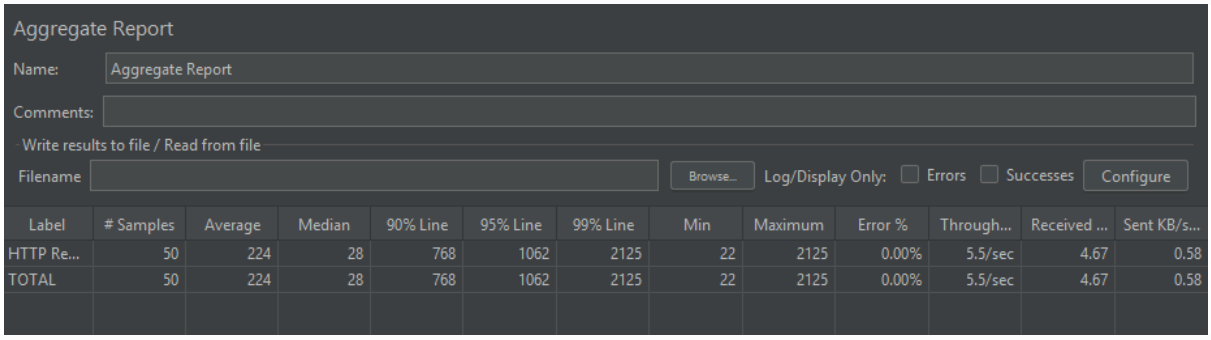
1] Execute Test 1 — Light Load
Make sure the Thread Group contains:
Number of Threads = 10
Ramp-up Period = 10 seconds
Loop Count = 5
Step 1
Click the green **Start** ▶ button at the top of JMeter.
Step 2
JMeter starts the virtual users.
Step 3
Wait for the test to complete.
For one HTTP request:
10 × 5 = 50 requests

Step 4 View Test 1 Results
Click:
Summary Report
You will see something similar to:



Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB...	Sent KB/sec	Avg. Bytes
HTTP Requ...	50	224	22	2125	470.24	0.00%	5.5/sec	4.67	0.58	873.0
TOTAL	50	224	22	2125	470.24	0.00%	5.5/sec	4.67	0.58	873.0

Step 5 - Analyze Response Time
Click:
Aggregate Report
Look at:



Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Received ...	Sent KB/s...
HTTP Re...	50	224	28	768	1062	2125	22	2125	0.00%	5.5/sec	4.67	0.58
TOTAL	50	224	28	768	1062	2125	22	2125	0.00%	5.5/sec	4.67	0.58

Step 6 - Analyze Throughput
In the **Summary Report** or **Aggregate Report**, find:
Throughput
Throughput indicates how many requests the application processes per unit of time.
For example:
Throughput = 328 /min
means the test achieved approximately 6 requests per second.

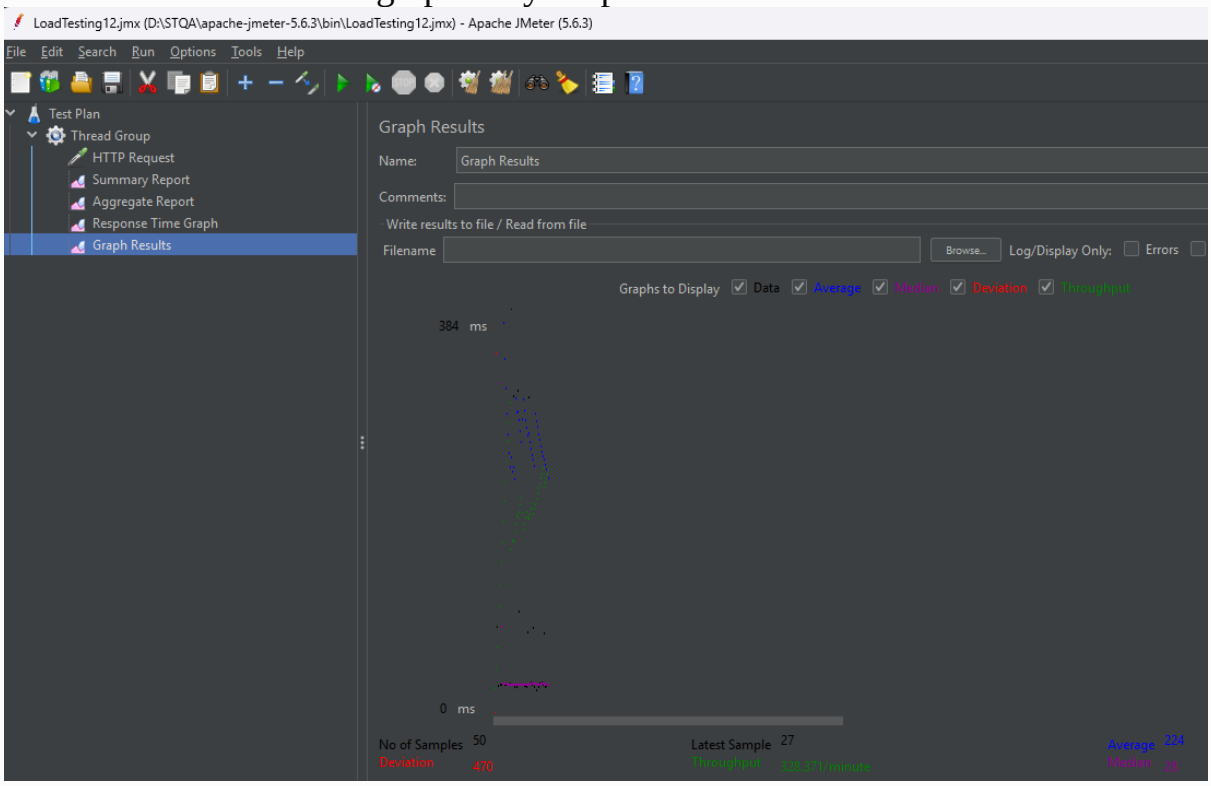
Step 7 - Analyze Error Percentage
Find:
Error %
Ideally, unexpected errors should be:
0%
The formula is:
Error Percentage=Failed Requests/ Total Requests×100
For example:
Total Requests = 20
Failed Requests = 16
Then:

Error Percentage=80%

Step 8 - View Response Time Graph
Click:
Response Time Graph
Observe the graph.
You should check whether:

- Response time is stable.
- Response time increases over time.
- There are sudden spikes.
- Response time becomes significantly higher as load increases.

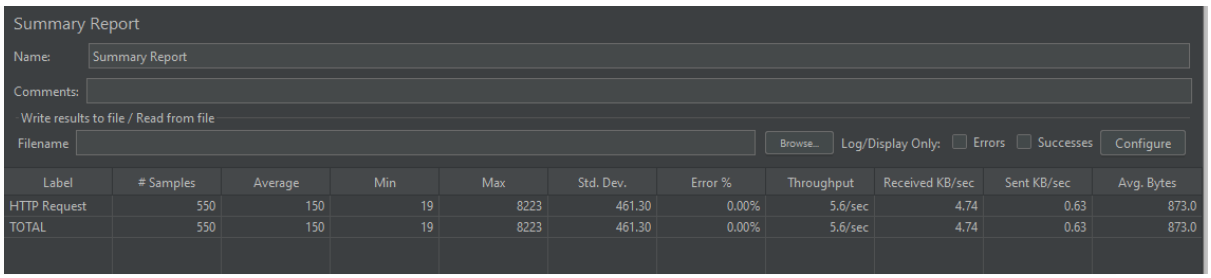
Take a screenshot of the graph for your practical record.



2] Execute Test 2 — Medium Load
Now change the Thread Group configuration.
Click **Thread Group**.
Enter:
Number of Threads = 50
Ramp-up Period = 30
Loop Count = 10
Therefore:
50 × 10 = 500 requests
Run the test

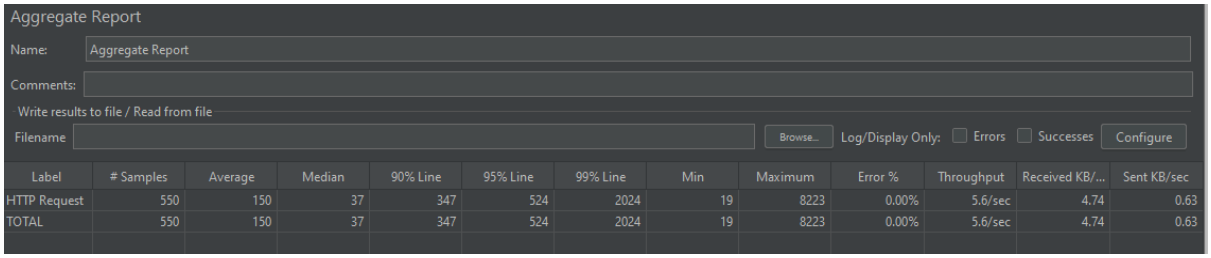
Click:
Start ▶
Wait until the test completes.
Then check: Record the results.

- Summary Report



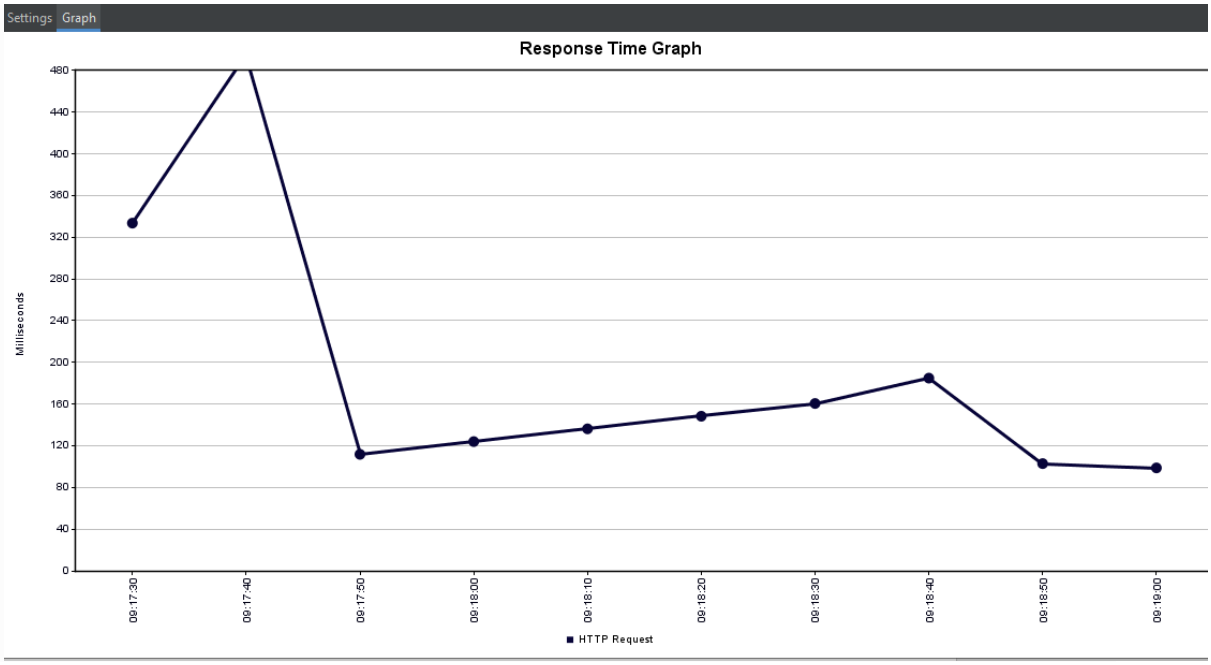
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	550	150	19	8223	461.30	0.00%	5.6/sec	4.74	0.63	873.0
TOTAL	550	150	19	8223	461.30	0.00%	5.6/sec	4.74	0.63	873.0

- Aggregate Report

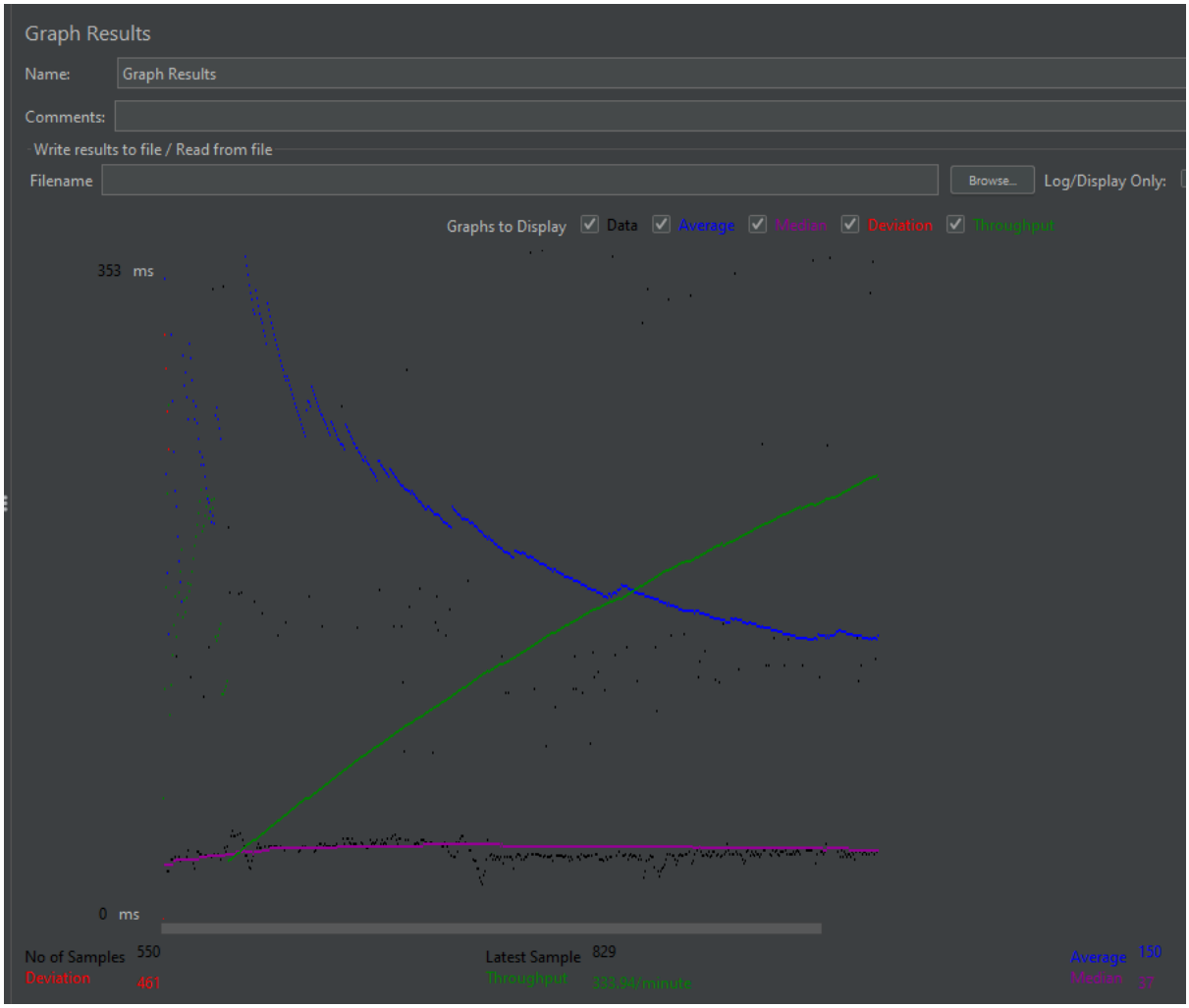


Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
HTTP Request	550	150	37	347	524	2024	19	8223	0.00%	5.6/sec	4.74	0.63
TOTAL	550	150	37	347	524	2024	19	8223	0.00%	5.6/sec	4.74	0.63

- Response Time Graph



- Graph Results



3] Execute Test 3 — Heavy Load

Now change the Thread Group again.

Enter:

Number of Threads = 100

Ramp-up Period = 60 seconds

Loop Count = 10

Therefore:

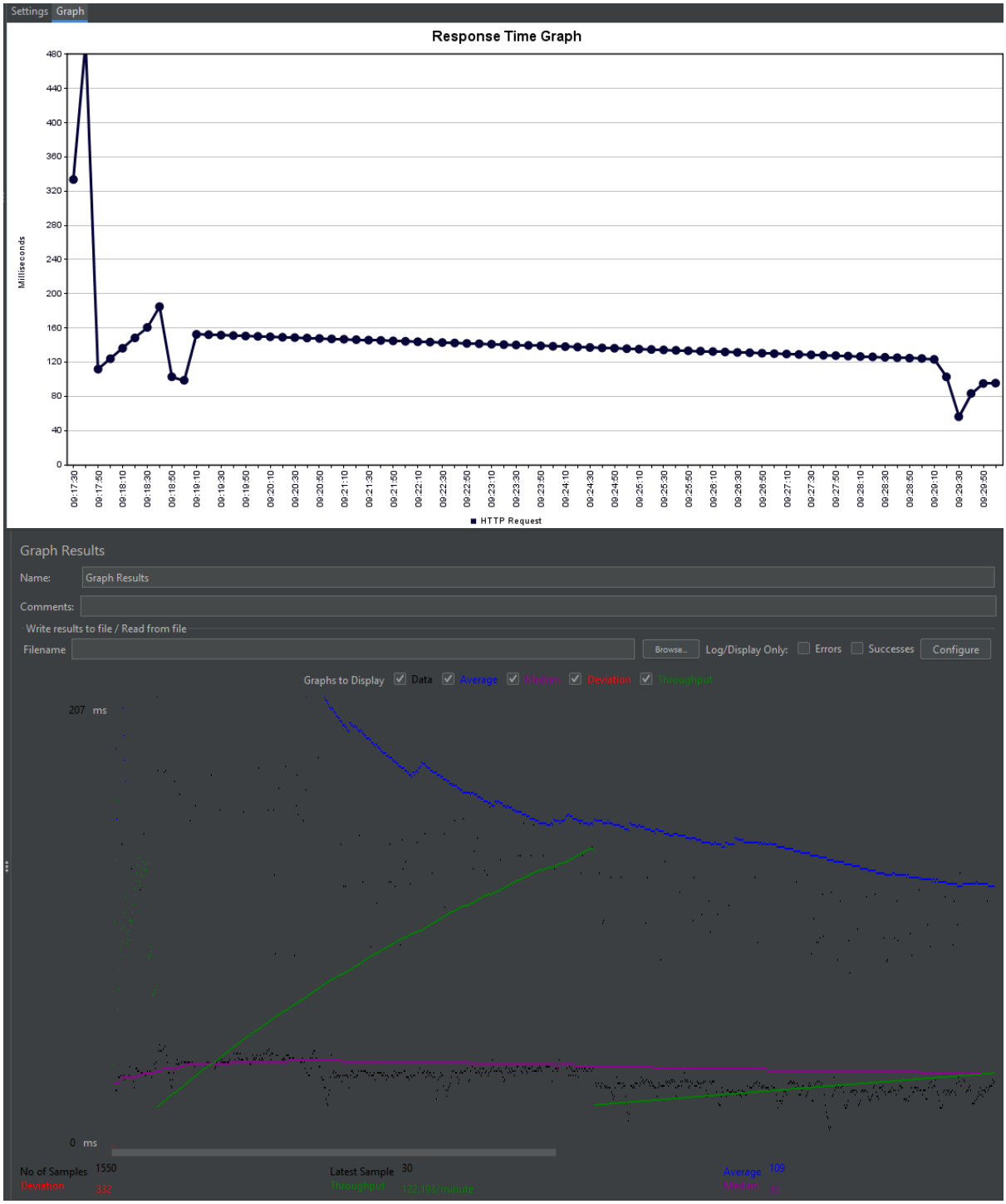
100 × 10 = 1,000 requests

Click:

Start ▶

Wait until the test completes.

Analyze the results.



Prepare the Final Results Table

Use this table in your practical record:

Enter the actual values generated by your JMeter test.

Compare the Three Tests results.

Test	No. of Samples	Average Response Time	Median	Deviation	Throughput/min
Test 1	50	224 ms	28 ms	470 ms	388.37/min
Test 2	550	150 ms	37 ms	461 ms	333.94/min
Test 3	1550	109 ms	32 ms	332 ms	122.193/min

Response Time
If response time increases considerably, the application may be experiencing performance degradation under increased load.
Throughput
See whether throughput increases, remains stable, or starts leveling off.
Error Percentage
An increase in unexpected errors under heavier load is an important performance concern.

Generate an HTML Graphical Report on Windows
You can generate JMeter's HTML dashboard using **Windows Command**
Step 1
Save your test plan:
LoadTesting21.jmx
Step 2
Open the JMeter bin folder.
For example: D:\STQA\apache-jmeter-5.6.3\bin
Step 3
Click the File Explorer address bar.
Type: cmd
A Windows Command Prompt will open at that location.
Step 4
Run:
jmeter.bat -n -t "LoadTesting21.jmx" -l "results.jtl" -e -o "report"

Here:

```
D:\STQA\apache-jmeter-5.6.3\bin>jmeter.bat -n -t "LoadTesting21.jmx" -l "results.jtl" -e -o "report"
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
Aug 18, 2026 10:59:48 AM java.util.prefs.WindowsPreferences <init>
WARNING: Could not open/create prefs root node Software\JavaSoft\Prefs at root 0x80000002. Windows RegCreateKeyEx(...) returned error co
de 5.
Creating summariser <summary>
Created the tree successfully using LoadTesting21.jmx
Starting standalone test @ August 18, 2026 10:59:53 AM IST (1787030993354)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary +    111 in 00:00:07 =   16.6/s Avg:    20 Min:     7 Max:    583 Err:     0 (0.00%) Active: 1 Started: 12 Finished: 11
summary +    500 in 00:00:30 =   16.7/s Avg:    13 Min:     6 Max:     77 Err:     0 (0.00%) Active: 1 Started: 62 Finished: 61
summary =    611 in 00:00:37 =   16.7/s Avg:    14 Min:     6 Max:    583 Err:     0 (0.00%)
summary +    389 in 00:00:23 =   17.0/s Avg:    13 Min:     6 Max:     52 Err:     0 (0.00%) Active: 0 Started: 100 Finished: 100
summary =   1000 in 00:01:00 =   16.8/s Avg:    14 Min:     6 Max:    583 Err:     0 (0.00%)
Tidying up ...    @ August 18, 2026 11:00:53 AM IST (1787031053067)
... end of run

D:\STQA\apache-jmeter-5.6.3\bin>
```

Option	Meaning
-n	Run without JMeter GUI
-t	Specify test plan
-l	Store results
-e	Generate HTML dashboard
-o	Specify report folder

Step 5

After completion, open:

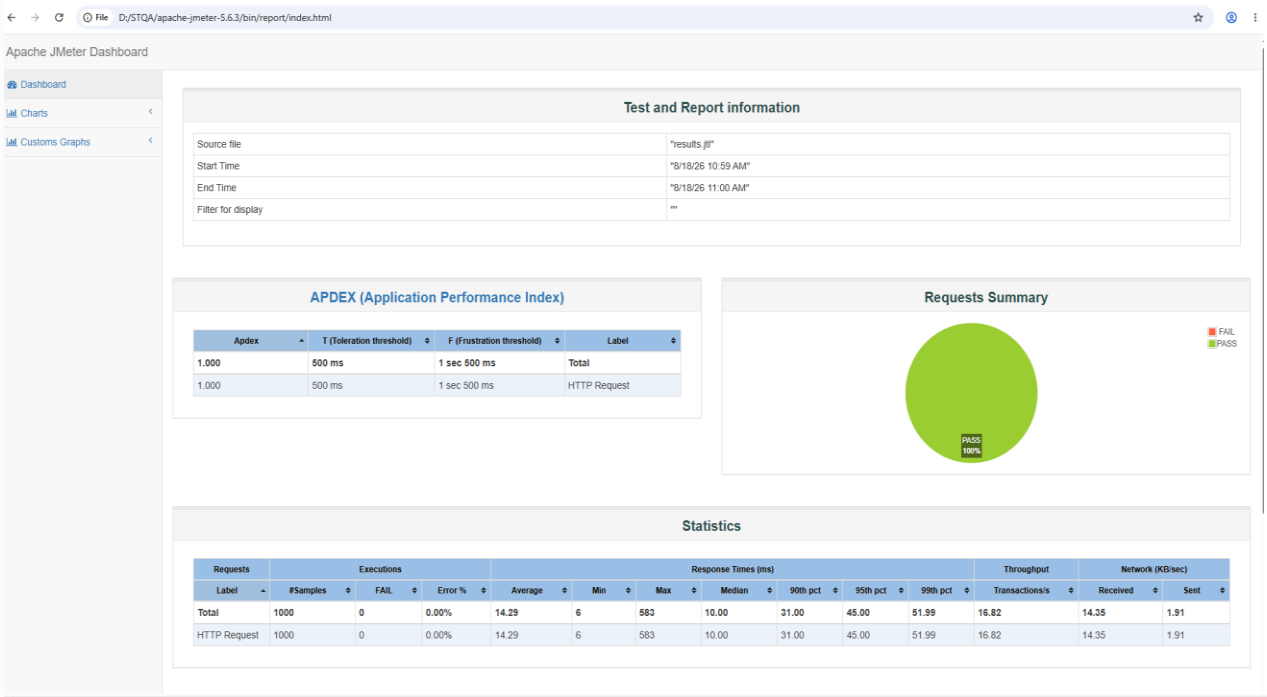
```
D:\STQA\apache-jmeter-5.6.3\bin>D:\STQA\apache-jmeter-5.6.3\bin\report\index.html
```

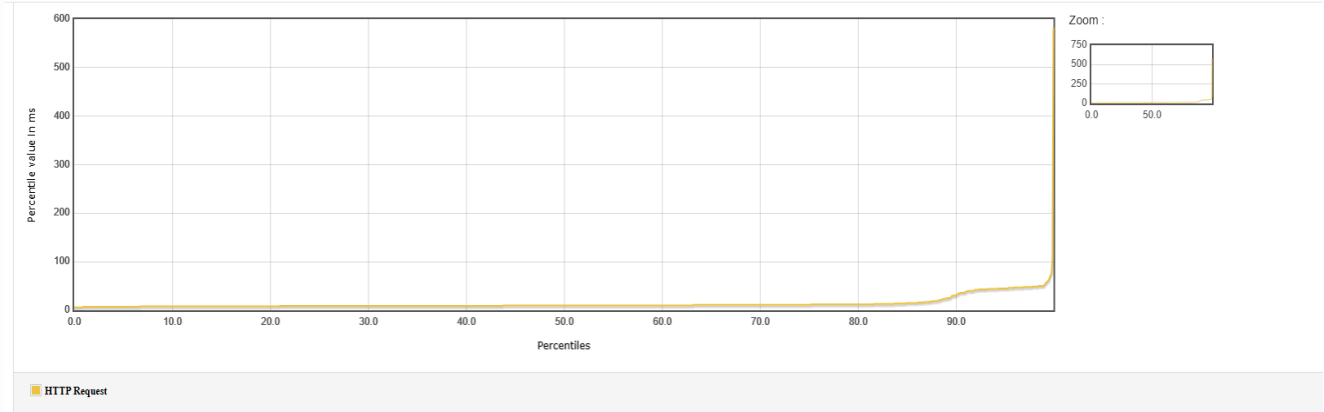
It will open in your browser.

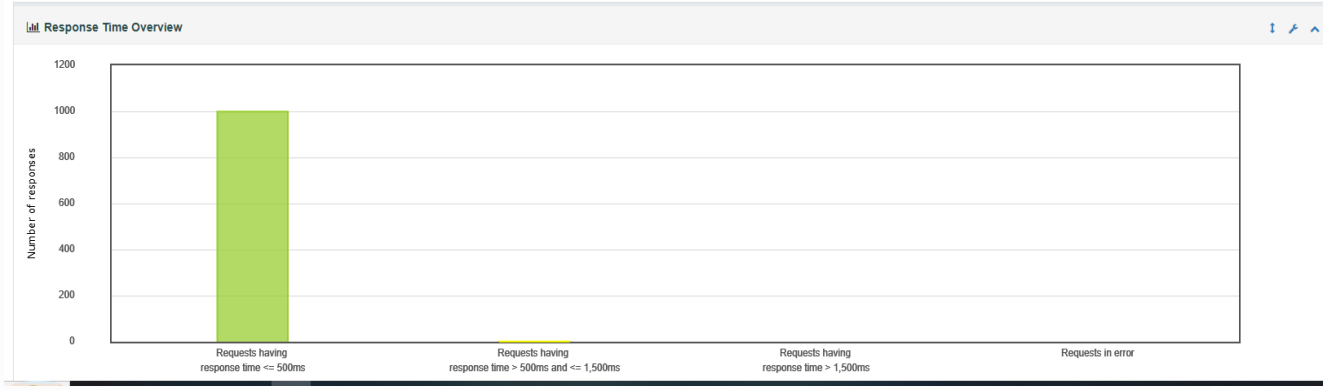
Graphical Report

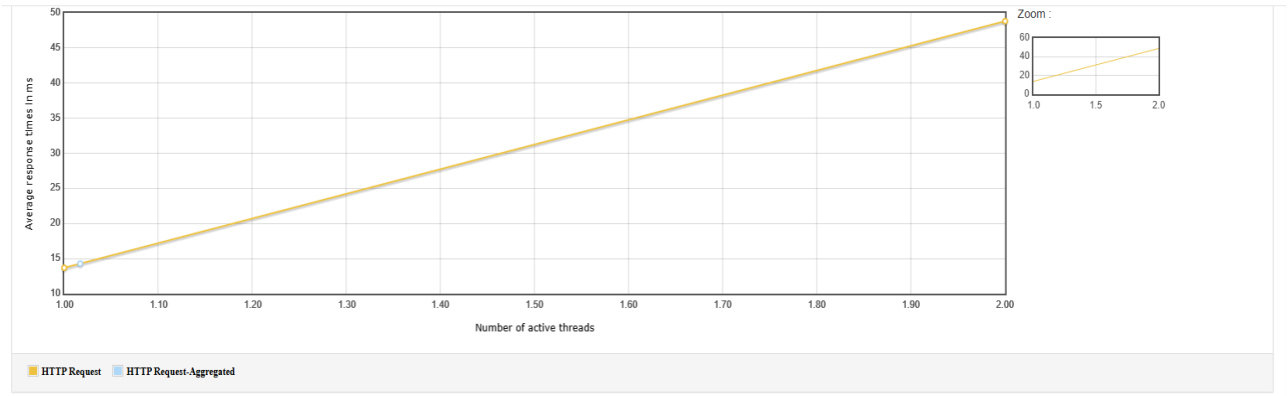
The HTML dashboard provides information about:

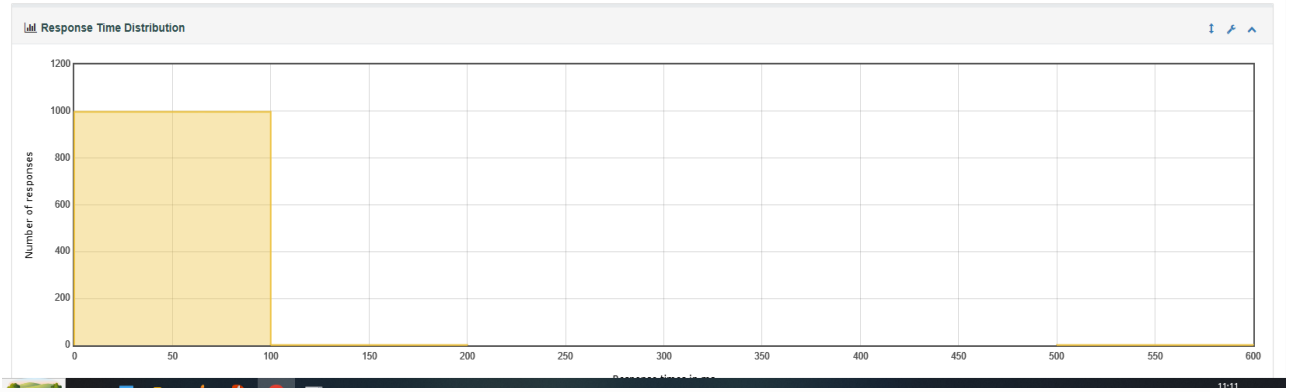
- Response time
- Response-time percentiles
- Throughput
- Errors
- Active threads
- Response-time distribution
- Request statistics


File D:/STQA/apache-jmeter-5.6.3/bin/report/index.html
Apache JMeter Dashboard
Dashboard
Charts
Customs Graphs
Test and Report information
Source file "results.jtl"
Start Time "8/18/26 10:59 AM"
End Time "8/18/26 11:00 AM"
Filter for display ""
APDEX (Application Performance Index)
Apdex | T (Toleration threshold) | F (Frustration threshold) | Label
1.000 | 500 ms | 1 sec 500 ms | Total
1.000 | 500 ms | 1 sec 500 ms | HTTP Request
Requests Summary
FAIL
PASS
PASS 100%
Statistics
Requests | Executions | | | Response Times (ms) | | | | | | | Throughput | Network (KB/sec)
Label | #Samples | FAIL | Error % | Average | Min | Max | Median | 90th pct | 95th pct | 99th pct | Transactions/s | Received | Sent
Total | 1000 | 0 | 0.00% | 14.29 | 6 | 583 | 10.00 | 31.00 | 45.00 | 51.99 | 16.82 | 14.35 | 1.91
HTTP Request | 1000 | 0 | 0.00% | 14.29 | 6 | 583 | 10.00 | 31.00 | 45.00 | 51.99 | 16.82 | 14.35 | 1.91


Percentile value in ms
Percentiles
Zoom :
HTTP Request


Response Time Overview
Number of responses
Requests having response time <= 500ms
Requests having response time > 500ms and <= 1,500ms
Requests having response time > 1,500ms
Requests in error





Final Practical Observation

Write your observation based on your actual results.
For example:
The load testing results were compared for 10, 50, and 100 virtual users. As the number of users increased, the response time and throughput were observed and the error percentage was recorded. The graphical reports helped identify changes in application performance under different load conditions.
Do **not** claim specific performance values unless they are actually obtained from your JMeter run.

27. Result

The web application was successfully load-tested using Apache JMeter on Windows. Different thread counts, ramp-up times, and loop counts were configured. Response time, throughput, and error percentage were measured and analyzed using Summary Report, Aggregate Report, Response Time Graph, and the HTML graphical dashboard.